Vector – Implementation

For the test with the moped tool, the **Vector** class must be implemented in a single header file (**vector.h**). This means that the usual separation of interface (.h) and implementation (.cpp) is no longer necessary. By limiting the implementation to one header file, the transition to the template version of the **vector** class is simplified.

The version uploaded for the test may only include the functionality described here, extended by iterators and templates that will be developed in the following units. In particular, your code must not contain any additional practice examples or examples from previous semester tests. If you want to practise, save a basic version that you can use for the upload.

Please note the deadline for uploading your vector implementation on **13.11.2026**

You should test your implementation thoroughly and possibly also design your own test routines (e.g. `main` functions).

We provide weekly test files for support, but we cannot give a **guarantee** that your vector class is correct.

1 Basic functionality

1.1 Instance Variables

The `Vector` class has the following **instance variables**:

size_t sz: Contains the number of elements stored in the **Vector**.

size_t max_sz: Contains the maximum possible number of elements (capacity of the **Vector**).

double* values: Points to a (dynamically allocated) array in which the **Vector** stores the elements.

hints:

You can define a minimum for the capacity of your **Vector**, which is defined in an additional class variable **static constexpr size_t min_capacity**. For example, if **min_capacity** = 5, the capacity of the **vector** (**max_sz**) must be at least 5 at all times. Whether you specify a minimum capacity or allow **vector**s whose capacity is 0 is just a matter of personal taste in the context of this course. Both approaches have their advantages and disadvantages and lead to different special cases, which must be handled accordingly in the code. It is advantageous to define a type alias **using value_type = double;** and to use this instead of **double** wherever it fits. This makes the transition to templates easier later on.

1.2 Constructors/Destructor

The `Vector` class has the following **constructors and a destructor**:

Default constructor: Returns an empty **Vector**.

Copy constructor: Returns a **Vector** with the same content as the parameter.

Constructors with the following parameter lists:

(size_t n): Creates an empty **Vector** with space for n elements.

(std::initializer_list<double>): Creates a **Vector** with the specified content.

Destructor: The destructor must free allocated memory again.

Avoid memory leaks and pay attention to the correct handling of special cases such as Vector(0) and Vector{}. To find problems with memory allocation, `valgrind` is a helpful tool that can be easily installed on the virtual machine.

1.3 Member Functions

The `Vector` class has the following **member functions**:

copy assignment operator: The `this` object takes a copy of the content of the parameter (necessary due to dynamic memory allocation).

size_t size() const: Returns the number of currently stored elements.

bool empty() const: Returns `true` if the **vector** is empty, `false` otherwise.

void clear(): Deletes all elements from the **vector**.

void reserve(size_t n): If the capacity of the **vector** is not already at least n, it is increased accordingly.

void shrink_to_fit(): Capacity of the **Vector** is reduced to the required minimum (number of currently stored elements).

void push_back(double x): A copy of `x` is added to the **Vector** as the last element (at the end).

void pop_back(): Removes the last element from the **Vector**. Throws an exception of type `std::runtime_error` if the **vector** was empty.

double& operator[](size_t index): Returns a reference to the element at the specified position (index). If the index is not within the permitted range, an exception of type `std::runtime_error` must be thrown.

const double& operator[](size_t index) const: Returns a reference to the element at the specified position (index). If the index is not within the permitted range, an exception of type `std::runtime_error` must be thrown. (Version for const **vector**s)

size_t capacity() const: Returns the current capacity of the **vector**.

1.4 Output Format

An output operator must be implemented for the `Vector` class.

ostream& operator<<(ostream&, const Vector&): outputs [element1, element2, element3].

For example Vector x{1,2,3,4} → [1, 2, 3, 4]

The use of **friend** is permitted for the implementation of **operator<<**.

2 Iterators

To use the **Vector** with STL algorithms, iterators and some type aliases are required. These are best defined at the beginning of the `Vector` class. Make sure that you only use the appropriate aliases in your **Vector** class instead of the underlying types.

```
class Vector{
public:
  class ConstIterator;
  class Iterator;
  using value_type = double;
  using size_type = std::size_t;
  using difference_type = std::ptrdiff_t;
  using reference = value_type&;
  using const_reference = const value_type&;
  using pointer = value_type*;
  using const_pointer = const value_type*;
  using iterator = Vector::Iterator;
  using const_iterator = Vector::ConstIterator;
private:
  //Instance variables
public:
  //Member Functions
class Iterator {
    public:
      using value_type = Vector::value_type;
      using reference = Vector::reference;
      using pointer = Vector::pointer;
      using difference_type = Vector::difference_type;
      using iterator_category = std::forward_iterator_tag;
    private:
    //Instance variables
    public:
    //Member Functions
  };
  class ConstIterator {
    public:
      using value_type = Vector::value_type;
      using reference = Vector::const_reference;
      using pointer = Vector::const_pointer;
      using difference_type = Vector::difference_type;
      using iterator_category = std::forward_iterator_tag;
    private:
    //Instance variables
    public:
    //Member Functions
  };
};
```

2.1 Extension of the class Vector

Extend the **Vector** class with **begin()** and **end()** member functions.

iterator begin(): Returns an iterator that refers to the first element in the **Vector**, or is equal to the end iterator if the **Vector** is empty.

iterator end(): Returns an iterator that refers to the virtual element after the last element contained in the **vector**.

const_iterator begin() const: As begin() for constant **vector** elements.

const_iterator end() const: As end() for constant **vector** elements.

2.2 Iterator

The class **Iterator** has the following **instance variables**:

pointer ptr: Points to the corresponding element in **vector**.

The class **Iterator** has the following **Constructors**:

Default: Creates an iterator with the value **nullptr** for the instance variable **ptr**.

Parameter list (pointer ptr): Creates an iterator in which the instance variable **ptr** is set to the value of the parameter.

The class **Iterator** has the following **member functions**: Which methods should be **const** methods?

reference operator*() const?: Returns a reference to the value to which the iterator refers (to which the instance variable **ptr** points).

pointer operator->() const?: Returns a pointer to the value referenced by the iterator.

bool operator==(const const_iterator&) const?: Compares whether the two pointers are the same. (A global function might be a better choice.)

bool operator!=(const const_iterator&) const?: Compares whether the two pointers are different. (A global function might be a better choice.)

iterator& operator++() const?: (Prefix) Iterator is switched to the next element in the **Vector**. The method returns a reference to the changed iterator.

iterator operator++(int) const?: (Postfix) Iterator is switched to the next element in **Vector**. A copy of the original iterator is returned.

operator const_iterator() const?: (type conversion) Allows the conversion of **Iterator** to **ConstIterator**.

2.3 ConstIterator

The class **ConstIterator** has the following **instance variables**:

pointer ptr: Points to the corresponding element in the **Vector**.

The class **ConstIterator** has the following **constructors**:

Default: Creates a ConstIterator with the value **nullptr** for the instance variable **ptr**.

Parameter list (pointer ptr): Creates a ConstIterator in which the instance variable **ptr** is set to the value of the parameter.

The class **ConstIterator** has the following **member functions**. Which methods should be **const** methods?

reference operator*() const?: Returns a reference to the value to which the iterator refers (to which the instance variable **ptr** points).

pointer operator->() const?: Returns a pointer to the value referenced by the ConstIterator.

bool operator==(const const_iterator&) const?: Compares whether the two pointers are the same. (A global function might be a better choice.)

bool operator!=(const const_iterator&) const?: Compares whether the two pointers are different. (A global function might be a better choice.)

const_iterator& operator++() const?: (Prefix) ConstIterator is switched to the next element in **Vector**. The method returns a reference to the changed ConstIterator.

const_iterator operator++(int) const?: (Postfix) ConstIterator is switched to the next element in **Vector**. A copy of the original ConstIterator is returned.

2.4 Member functions `insert` and `erase`

The member functions **insert** and **erase** can be copied from here:

```
iterator insert(const_iterator pos, const_reference val) {
  auto diff = pos-begin();
  if (diff<0 || static_cast<size_type>(diff)>sz)
    throw std::runtime_error("Iterator out of bounds");
  size_type current{static_cast<size_type>(diff)};
  if (sz>=max_sz)
    reserve(max_sz*2); //Attention special case, if no minimum size is defined
  for (auto i {sz}; i-->current;)
    values[i+1]=values[i];
  values[current]=val;
  ++sz;
  return iterator{values+current};
}

iterator erase(const_iterator pos) {
  auto diff = pos-begin();
  if (diff<0 || static_cast<size_type>(diff)>=sz)
    throw std::runtime_error("Iterator out of bounds");
  size_type current{static_cast<size_type>(diff)};
  for (auto i{current}; i<sz-1; ++i)
    values[i]=values[i+1];
  --sz;
  return iterator{values+current};
}
```

For **insert** and **erase** to work in this way, the following method must also be implemented:

```
friend Vector::difference_type operator-(const Vector::ConstIterator& lop,
                                         const Vector::ConstIterator& rop) {
  return lop.ptr-rop.ptr;
}
```

3 Templates

To convert your **Vector** class into a template class, we recommend the following procedure:

1. Class **Vector** becomes a template with a type parameter

2. Replace the data type `double` in the appropriate places with `T` (much easier if the type aliases have already been used, otherwise it is a good idea to do this now).

3. The definitions of the template functions must also be in vector.h, as they are required for instantiation by the compiler. The easiest way is to define them inline (within the class definitions).

4. Test with different data types and correct errors if necessary.